LINUX ADMINISTRATION PRACTICAL LAB MANUAL

Executable Commands & System Operations Guide for Red Hat Enterprise Linux (RHEL)

MODULE 2 — MANAGING STORAGE & NETWORK SERVICES

Practical No. 7: Managing Storage

a. Understanding MBR and GPT Partitions

`sudo fdisk -l`
Lists disks and shows partition table type (dos MBR, gpt GPT).

`sudo parted -l`
Lists disks with partition table type shown explicitly.

`sudo blkid`
Shows block device UUIDs and filesystem types.

`lsblk`
Shows block device tree.

b. Managing Partitions and File Systems

`sudo fdisk /dev/sdX`
Create/delete MBR partitions (interactive n, d, p, w).

`sudo gdisk /dev/sdX`
Create/delete GPT partitions (interactive).

`sudo parted /dev/sdX`
Create/manage partitions (MBR or GPT).

`sudo mkfs.ext4 /dev/sdX1`
Create an ext4 file system.

`sudo mkfs.xfs /dev/sdX1`
Create an xfs file system.

`sudo mkswap /dev/sdX2`
Create a swap partition.

c. Mounting File Systems

`sudo mkdir /mnt/data`
Create mount point.

`sudo mount /dev/sdX1 /mnt/data`
Mount a file system.

`mount | grep sdX1`
Verify mount.

`sudo umount /mnt/data`
Unmount a file system.

`sudo blkid /dev/sdX1`
Get UUID for fstab entry.

`echo 'UUID=<uuid> /mnt/data ext4 defaults 0 0' | sudo tee -a /etc/fstab`
Append entry to /etc/fstab.

`sudo mount -a`
Mount all entries in /etc/fstab (test fstab syntax).

d. List, Create, Delete Partitions on MBR and GPT Disks

`sudo fdisk -l /dev/sdX`
List partitions on the disk.

`sudo parted /dev/sdX print`
List partitions.

`sudo parted /dev/sdX rm 1`
Delete partition number 1.

`sudo partprobe /dev/sdX`
Reload partition table without reboot.

e. Configure Systems to Mount File Systems at Boot by Universally Unique ID (UUID) or Label

`sudo blkid /dev/sdX1`
Get the UUID of a partition.

`sudo e2label /dev/sdX1 mydata`
Set a label on an ext4 partition.

`sudo xfs_admin -L mydata /dev/sdX1`
Set a label on an xfs partition.

`sudo mount -a`
Test fstab without rebooting.

`findmnt /mnt/data`
Verify the mount.

f. Add New Partitions, Logical Volumes, and Swap to a System Non-Destructively

`sudo parted /dev/sdX mkpart primary 0% 100%`
Create new partition (non-destructive: use free space only).

`sudo partprobe /dev/sdX`
Reload partition table.

`sudo pvcreate /dev/sdX1`
Create physical volume.

`sudo vgcreate myvg /dev/sdX1`
Create volume group.

`sudo vgextend myvg /dev/sdY1`
Add more space to existing VG.

`sudo lvcreate -n mylv -L 10G myvg`
Create a logical volume.

`sudo lvextend -L +5G /dev/myvg/mylv`
Grow a logical volume.

`sudo resize2fs /dev/myvg/mylv`
Resize ext4 after lvextend.

`sudo xfs_growfs /mnt/data`
Resize xfs after lvextend (mounted).

`sudo fallocate -l 2G /swapfile`
Allocate swap file size.

`sudo chmod 600 /swapfile`
Set strict file permissions for swap file.

`sudo mkswap /swapfile`
Set up swap space on file.

`sudo swapon /swapfile`
Enable swap file.

```
echo '/swapfile swap swap defaults 0 0' | sudo tee -a /etc/fstab
```
Make swap persistent in fstab.

```
swapon --show
```
Verify active swap.

g. Create, Mount, Unmount, and Use vfat, ext4, and xfs File Systems

```
sudo mkfs.vfat -F32 /dev/sdX1
```
Create a vfat (FAT32) file system.

```
sudo mkfs.ext4 /dev/sdX2
```
Create an ext4 file system.

```
sudo mkfs.xfs /dev/sdX3
```
Create an xfs file system.

```
sudo mkdir -p /mnt/vfatdata /mnt/ext4data /mnt/xfsdata
```
Create mount points for filesystems.

```
sudo mount -t vfat /dev/sdX1 /mnt/vfatdata
```
Mount vfat file system.

```
sudo mount -t ext4 /dev/sdX2 /mnt/ext4data
```
Mount ext4 file system.

```
sudo mount -t xfs /dev/sdX3 /mnt/xfsdata
```
Mount xfs file system.

```
df -hT
```
Show mounted file systems and types.

```
mount | grep sdX
```
Confirm mount options.

```
sudo umount /mnt/vfatdata
```
Unmount vfat file system.

```
sudo umount /mnt/ext4data
```
Unmount ext4 file system.

```
sudo umount /mnt/xfsdata
```
Unmount xfs file system.

Practical No. 8: Configuring Networking

a. Managing Network Addresses and Interfaces

`ip addr show`
Displays all active network interfaces along with their currently assigned IP addresses and subnet masks.

`ip link show`
Shows the operational state, link status, and MAC hardware addresses for all network interfaces.

b. Validating Network Configuration

`ip route show`
Displays the kernel routing table, active gateways, and network paths configured for packet forwarding.

`ping -c 4 8.8.8.8`
Sends four ICMP echo request packets to verify network connectivity and packet transmission to a remote host.

c. Managing Network Configuration with nmtui and nmcli

`nmtui`
Launches an interactive, menu-driven text user interface inside the terminal to configure Network Manager connections.

`nmcli connection show`
Lists all saved NetworkManager connection profiles, their corresponding UUIDs, connection types, and active devices.

`nmcli device status`
Displays the status, device type, connection profile, and NetworkManager management state for each hardware device.

d. Configure IPv4 and IPv6 Addresses

`nmcli con mod "ens33" ipv4.addresses 192.168.1.100/24 ipv4.gateway 192.168.1.1 ipv4.method manual`
Sets a static IPv4 address, netmask, and default gateway while switching interface addressing mode from DHCP to manual.

`nmcli con mod "ens33" ipv6.addresses 2001:db8::10/64 ipv6.method manual`
Assigns a static IPv6 address with prefix length and configures IPv6 management mode to manual.

`nmcli con up "ens33"`
Restarts the specified connection profile to apply and activate newly configured IP parameters.

Practical No. 9: Configuring SSH

a. Using SSH and Related Utilities

```
sudo systemctl enable --now sshd
```
Enables and starts the SSH daemon service.

```
scp file.txt user@192.168.1.50:/tmp/
```
Securely copies a file to a remote host over SSH.

```
sftp user@192.168.1.50
```
Initiates an interactive secure file transfer session with a remote host.

b. Access Remote Systems Using SSH

```
ssh -o StrictHostKeyChecking=no user@192.168.1.50
```
Connects to a remote host via SSH while bypassing host key verification prompts.

```
ssh user@192.168.1.50 "hostname; uptime"
```
Executes a remote command directly on the target system without opening an interactive shell session.

c. Configuring Key-Based Authentication with Passphrases

```
ssh-keygen -t rsa -b 2048 -f ~/.ssh/id_rsa -N ""
```
Generates a new RSA SSH public/private key pair non-interactively.

```
eval $(ssh-agent -s)
```
Starts the SSH authentication agent in the current shell session.

```
ssh-add ~/.ssh/id_rsa
```
Adds the generated private SSH key to the active authentication agent.

d. Configure Key-Based Authentication for SSH

```
ssh-copy-id -o StrictHostKeyChecking=no user@192.168.1.50
```
Copies the local SSH public key to the remote host for passwordless authentication.

```
chmod 700 ~/.ssh && chmod 600 ~/.ssh/authorized_keys 2>/dev/null || chmod 700 ~/.ssh
```
Secures SSH directory and authorized keys file permissions.

```
ssh -i ~/.ssh/id_rsa user@192.168.1.50 "whoami; echo Login Successful without Password"
```
Verifies passwordless key-based authentication by logging into the remote host.

Practical No. 10: Configuring Web Server

b. Understanding Apache Configuration Files

```
grep -v '^[[:space:]]*#' /etc/httpd/conf/httpd.conf | grep -v '^[[:space:]]*$' | head -n 15
```

Inspects active primary Apache configuration directives by filtering out comments and blank lines.

```
apachectl configtest
```

Verifies Apache configuration syntax integrity.

```
grep -E "^(Listen|IncludeOptional)" /etc/httpd/conf/httpd.conf
```

Displays default listening ports and configuration include paths.

c. Creating Apache Virtual Host

```
sudo mkdir -p /var/www/site1.example.com/html
```

Creates a dedicated document root directory for a virtual host.

```
echo "<h1>Site 1: Virtual Host configured successfully</h1>" | sudo tee /var/www/site1.example.com/html/index.html
```

Creates a custom landing page for the virtual host site.

```
sudo chown -R apache:apache /var/www/site1.example.com && sudo restorecon -Rv /var/www/site1.example.com
```

Sets appropriate ownership and restores SELinux security contexts for the virtual host directory.

```
sudo tee /etc/httpd/conf.d/site1.example.com.conf << 'EOF' ... EOF
```

Creates the Apache Virtual Host configuration file for the domain.

```
echo "127.0.0.1 site1.example.com" | sudo tee -a /etc/hosts
```

Adds a local host mapping entry for domain resolution testing.

```
apachectl configtest && sudo systemctl reload httpd
```

Validates configuration syntax and reloads the Apache service to apply changes.

```
curl http://site1.example.com
```

Tests the virtual host configuration via curl.

Practical No. 11: Configure Firewall

a. Linux Firewall with iptables

`sudo dnf install -y iptables iptables-services`
Installs iptables legacy utilities and firewall services.

`sudo iptables -L -n -v --line-numbers`
Lists current iptables rules with packet counters, verbose details, and line numbers.

`sudo iptables -A INPUT -i lo -j ACCEPT && sudo iptables -A INPUT -m conntrack --cts tate ESTABLISHED,RELATED -j ACCEPT`
Allows loopback interface traffic and established/related connection packets.

`sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT && sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT`
Appends rules to permit incoming SSH (port 22) and HTTP (port 80) traffic.

`sudo iptables -A INPUT -p icmp --icmp-type echo-request -j REJECT`
Blocks incoming ICMP ping echo requests.

`sudo iptables-save | sudo tee /etc/sysconfig/iptables`
Saves active iptables rules persistently to the system configuration file.

b. Working with Firewall (firewalld)

`sudo firewall-cmd --state`
Checks the running state of the firewalld service.

`sudo firewall-cmd --get-default-zone && sudo firewall-cmd --list-all`
Identifies the default active firewall zone and lists all assigned services, ports, and rules.

`sudo firewall-cmd --permanent --add-service=http && sudo firewall-cmd --permanent --add-service=https`
Permanently allows HTTP and HTTPS services through firewalld.

`sudo firewall-cmd --permanent --add-port=8080/tcp`
Permanently opens custom TCP port 8080.

`sudo firewall-cmd --reload`
Reloads the firewall configuration to apply pending changes.

`sudo firewall-cmd --list-services && sudo firewall-cmd --list-ports`
Verifies currently active firewall services and ports.

Practical No. 12: Accessing Network Storage

b. Using CIFS Services

```
smbclient -L //localhost -N
```

Queries available CIFS/Samba shares using guest access without a password.

```
sudo mkdir -p /mnt/cifs_client && sudo mount -t cifs //localhost/cifs_share /mnt/cifs_client -o guest,vers=3.0
```

Creates a mount point and mounts a CIFS share using guest credentials.

```
df -hT /mnt/cifs_client && cat /mnt/cifs_client/cifs_data.txt
```

Verifies the mounted CIFS filesystem details and accesses test file contents.

c. Mounting Remote File Systems Through fstab

```
sudo umount /mnt/nfs_client 2>/dev/null && sudo umount /mnt/cifs_client 2>/dev/null
```

Unmounts temporary client mount points cleanly.

```
sudo tee -a /etc/fstab << 'EOF' ... EOF
```

Appends persistent NFS and CIFS remote mount entries into the /etc/fstab file.

```
sudo systemctl daemon-reload && sudo mount -a && df -hT | grep -E '(nfs|cifs)'
```

Reloads systemd, tests mounting all fstab entries, and verifies active network mounts.

d. Using Automount to Mount Remote File Systems

```
sudo umount /mnt/nfs_client 2>/dev/null && sudo umount /mnt/cifs_client 2>/dev/null
```

Unmounts static mounts to avoid conflicts with automount.

```
echo "/misc /etc/auto.misc --timeout=60" | sudo tee -a /etc/auto.master
```

Configures the master map entry for autofs in /etc/auto.master.

```
echo "nfs_share -fstype=nfs4,rw,soft,intr localhost:/" | sudo tee -a /etc/auto.misc
```

Defines an on-demand NFS map entry in /etc/auto.misc.

```
sudo systemctl enable --now autofs
```

Enables and starts the autofs daemon service.

```
ls -la /misc/nfs_share && cat /misc/nfs_share/nfs_data.txt && df -hT /misc/nfs_share
```

Triggers on-demand mounting via autofs, reads file contents, and verifies filesystem status.

Practical No. 13: Creating Shell scripts

a. Automate or Monitor Tasks/Processes

`sudo dnf install -y vim-enhanced`
Installs the enhanced Vim text editor package.

`vim monitor_process.sh`
Creates and opens a process monitoring and service recovery script using Vim.

`chmod +x monitor_process.sh && ./monitor_process.sh`
Makes the script executable and runs the process monitor.

`vim auto_backup_clean.sh`
Creates an automated backup and log archiving script in Vim.

`chmod +x auto_backup_clean.sh && ./auto_backup_clean.sh`
Makes the backup script executable and runs it.

b. Getting and Setting System Information

`vim get_sys_info.sh`
Creates a script to gather system reports, uptime, disk, and memory usage.

`chmod +x get_sys_info.sh && ./get_sys_info.sh`
Makes the system info script executable and executes it.

`vim set_sys_info.sh`
Creates a script to configure system hostname, timezone, and kernel parameters.

`chmod +x set_sys_info.sh && ./set_sys_info.sh`
Makes the system configuration script executable and runs it.

Practical No. 14: Scheduling Tasks

a. Configuring Cron to Automate Recurring Tasks

`sudo dnf install -y cronie at`
Installs cronie and at scheduling packages.

`sudo systemctl enable --now crond atd`
Enables and starts cron and at daemon services.

`crontab -l`
Lists existing cron table entries for the current user.

```
(crontab -l 2>/dev/null; echo "* * * * * date >> /tmp/cron_test.log 2>&1") | cronta
b
```

Appends a recurring cron job entry to execute a command every minute.

```
sleep 65
```

Pauses execution to wait for cron execution intervals.

```
cat /tmp/cron_test.log
```

Views the output log generated by the scheduled cron task.

b. Configuring At to Schedule Future Tasks

```
echo "echo 'Task executed via at at \$(date)' >> /tmp/at_output.txt" | at now + 2 m
inutes
```

Schedules a one-time execution task to run 2 minutes in the future.

```
atq
```

Inspects the scheduled pending jobs queue in at.

```
at -c 1
```

Inspects the environment and commands of a specific scheduled at job.

c. Schedule Tasks Using At and Cron

```
cat /tmp/at_output.txt
```

Verifies the execution output of the scheduled at task.

```
vim schedule_maintenance.sh
```

Creates a maintenance script combining both at and cron tasks using Vim.

```
chmod +x schedule_maintenance.sh && ./schedule_maintenance.sh
```

Makes the maintenance script executable and runs it.

```
crontab -r
```

Removes the current user's crontab entries.